

MongoDB Basic Practice Assignment



Status

Screening

MongoDB Basic Practice Assignment

Title: Course Enrollment System using MongoDB

1. Introduction

MongoDB is a NoSQL database that stores data in a flexible, JSON-like format called documents. Unlike traditional relational databases, MongoDB does not require a fixed schema, which makes it easier to handle dynamic and real-world data.

This assignment focuses on performing basic CRUD operations (Create, Read, Update, Delete) and using query operators.

2. Database & Collection Setup

A database named `courseDB` is created to simulate a course enrollment system.

```
use courseDB
```

A collection named `students` is used to store student data.

```
C:\Users\ASMAAN>mongosh
Current Mongosh Log ID: 69f5c435f8b8295dfe3682d0
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:  8.2.7
Using Mongosh:  2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-04-29T20:11:53.084+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use courseDB
switched to db courseDB
courseDB> db.students
courseDB.students
```

3. Insert Operations

Insert One Document

```
db.students.insertOne({
  name: "Arun",
  age: 21,
  course: "MERN Stack",
  status: "ongoing",
  enrolledAt: new Date()
})
```

Insert Multiple Documents

```
db.students.insertMany([
  {
    name: "Priya",
    age: 22,
    course: "MERN Stack",
    status: "ongoing",
    enrolledAt: new Date()
  },
  {
    name: "Rahul",
    age: 20,
    course: "Python",
    status: "completed",
    enrolledAt: new Date()
  },
  {
    name: "Sneha",
    age: 23,
    course: "Java",
    status: "ongoing"
  }
])
```

```

courseDB.students
courseDB> db.students.insertOne({name: "Arun", age: 21, course: "MERN Stack", status: "ongoing", enrolledAt: new Date() })
{
  acknowledged: true,
  insertedId: ObjectId('69f5c47bf8b8295dfe3682d1')
}
courseDB> db.students.insertMany([
  {
    name: "Priya",
    age: 22,
    course: "MERN Stack",
    status: "ongoing",
    enrolledAt: new Date()
  },
  {
    name: "Rahul",
    age: 20,
    course: "Python",
    status: "completed",
    enrolledAt: new Date()
  },
  {
    name: "Sneha",
    age: 23,
    course: "Java",
    status: "ongoing"
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f5c499f8b8295dfe3682d2'),
    '1': ObjectId('69f5c499f8b8295dfe3682d3'),
    '2': ObjectId('69f5c499f8b8295dfe3682d4')
  }
}
courseDB> db.students.find()

```

4. Read Operations

Fetch All Documents

```
db.students.find()
```

Filter Students by Course

```
db.students.find({ course: "MERN Stack" })
```

Filter by Multiple Conditions

```

db.students.find({
  course: "MERN Stack",
  status: "ongoing"
})

```

```

courseDB> db.students.find()
[
  {
    _id: ObjectId('69f5c47bf8b8295dfe3682d1'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing',
    enrolledAt: ISODate('2026-05-02T09:31:39.257Z')
  },
  {
    _id: ObjectId('69f5c499f8b8295dfe3682d2'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing',
    enrolledAt: ISODate('2026-05-02T09:32:09.784Z')
  },
  {
    _id: ObjectId('69f5c499f8b8295dfe3682d3'),
    name: 'Rahul',
    age: 20,
    course: 'Python',
    status: 'completed',
    enrolledAt: ISODate('2026-05-02T09:32:09.784Z')
  },
  {
    _id: ObjectId('69f5c499f8b8295dfe3682d4'),
    name: 'Sneha',
    age: 23,
    course: 'Java',
    status: 'ongoing'
  }
]
courseDB> db.students.find({ course: "MERN Stack" })
[
  {
    _id: ObjectId('69f5c47bf8b8295dfe3682d1'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing',
    enrolledAt: ISODate('2026-05-02T09:31:39.257Z')
  },
  {
    _id: ObjectId('69f5c499f8b8295dfe3682d2'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing',
    enrolledAt: ISODate('2026-05-02T09:32:09.784Z')
  }
]

```

5. Update Operations

Update One Document

```

db.students.updateOne(
  { name: "Arun" },

```

```
    { $set: { status: "completed" } }  
  )
```

Update Multiple Documents

```
db.students.updateMany(  
  { course: "MERN Stack" },  
  { $set: { status: "completed" } }  
)
```

```

courseDB> db.students.updateOne(
|   { name: "Arun" },
|   { $set: { status: "completed" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
courseDB> db.students.updateMany(
|   { course: "MERN Stack" },
|   { $set: { status: "completed" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 1,
  upsertedCount: 0
}
courseDB> db.students.find()
[
  {
    _id: ObjectId('69f5c47bf8b8295dfe3682d1'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed',
    enrolledAt: ISODate('2026-05-02T09:31:39.257Z')
  },
  {
    _id: ObjectId('69f5c499f8b8295dfe3682d2'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'completed',
    enrolledAt: ISODate('2026-05-02T09:32:09.784Z')
  },
  {
    _id: ObjectId('69f5c499f8b8295dfe3682d3'),
    name: 'Rahul',
    age: 20,
    course: 'Python',
    status: 'completed',
    enrolledAt: ISODate('2026-05-02T09:32:09.784Z')
  },
  {
    _id: ObjectId('69f5c499f8b8295dfe3682d4'),
    name: 'Sneha',
    age: 23,
    course: 'Java',
    status: 'ongoing'
  }
]

```

6. Delete Operations

Delete One Document

```
db.students.deleteOne({ name: "Rahul" })
```

Delete All Documents

```
db.students.deleteMany({})
```

```
courseDB> db.students.deleteOne({ name: "Rahul" })
{ acknowledged: true, deletedCount: 1 }
courseDB> db.students.find()
[
  {
    _id: ObjectId('69f5c47bf8b8295dfe3682d1'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed',
    enrolledAt: ISODate('2026-05-02T09:31:39.257Z')
  },
  {
    _id: ObjectId('69f5c499f8b8295dfe3682d2'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'completed',
    enrolledAt: ISODate('2026-05-02T09:32:09.784Z')
  },
  {
    _id: ObjectId('69f5c499f8b8295dfe3682d4'),
    name: 'Sneha',
    age: 23,
    course: 'Java',
    status: 'ongoing'
  }
]
courseDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 3 }
courseDB> db.students.find()
```

7. Query Operators

Greater Than (\$gt)

```
db.students.find({ age: { $gt: 21 } })
```

```

courseDB> db.students.find({ age: { $gt: 21 } })
[
  {
    _id: ObjectId('69f5c615f8b8295dfe3682d5'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing',
    enrolledAt: ISODate('2026-05-02T09:38:29.027Z')
  },
  {
    _id: ObjectId('69f5c615f8b8295dfe3682d7'),
    name: 'Sneha',
    age: 23,
    course: 'Java',
    status: 'ongoing'
  }
]

```

Less Than (\$lt)

```
db.students.find({ age: { $lt: 22 } })
```

```

courseDB> db.students.find({ age: { $lt: 22 } })
[
  {
    _id: ObjectId('69f5c615f8b8295dfe3682d6'),
    name: 'Rahul',
    age: 20,
    course: 'Python',
    status: 'completed',
    enrolledAt: ISODate('2026-05-02T09:38:29.027Z')
  },
  {
    _id: ObjectId('69f5c619f8b8295dfe3682d8'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing',
    enrolledAt: ISODate('2026-05-02T09:38:33.247Z')
  }
]

```

In Operator (\$in)


```
db.students.find({
  course: { $in: ["MERN Stack", "Python"] }
})
```

```
courseDB> db.students.find({
|   course: { $in: ["MERN Stack", "Python"] }
| })
[
  {
    _id: ObjectId('69f5c615f8b8295dfe3682d5'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing',
    enrolledAt: ISODate('2026-05-02T09:38:29.027Z')
  },
  {
    _id: ObjectId('69f5c615f8b8295dfe3682d6'),
    name: 'Rahul',
    age: 20,
    course: 'Python',
    status: 'completed',
    enrolledAt: ISODate('2026-05-02T09:38:29.027Z')
  },
  {
    _id: ObjectId('69f5c619f8b8295dfe3682d8'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing',
    enrolledAt: ISODate('2026-05-02T09:38:33.247Z')
  }
]
```

AND Operator (`$and`)

```
db.students.find({
  $and: [
    { course: "MERN Stack" },
    { age: { $gt: 20 } }
  ]
})
```

```
]
})
```

```
courseDB> db.students.find({
|   $and: [
|     { course: "MERN Stack" },
|     { age: { $gt: 20 } }
|   ]
| })
[
  {
    _id: ObjectId('69f5c615f8b8295dfe3682d5'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing',
    enrolledAt: ISODate('2026-05-02T09:38:29.027Z')
  },
  {
    _id: ObjectId('69f5c619f8b8295dfe3682d8'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing',
    enrolledAt: ISODate('2026-05-02T09:38:33.247Z')
  }
]
```

OR Operator (`$or`)

```
db.students.find({
  $or: [
    { status: "completed" },
    { course: "Java" }
  ]
})
```

```

courseDB> db.students.find({
|   $or: [
|     { status: "completed" },
|     { course: "Java" }
|   ]
| })
[
  {
    _id: ObjectId('69f5c615f8b8295dfe3682d6'),
    name: 'Rahul',
    age: 20,
    course: 'Python',
    status: 'completed',
    enrolledAt: ISODate('2026-05-02T09:38:29.027Z')
  },
  {
    _id: ObjectId('69f5c615f8b8295dfe3682d7'),
    name: 'Sneha',
    age: 23,
    course: 'Java',
    status: 'ongoing'
  }
]

```

Exists Operator (`$exists`)

```

db.students.find({
  enrolledAt: { $exists: true }
})

```

```

courseDB> db.students.find({
|   enrolledAt: { $exists: true }
| })
[
  {
    _id: ObjectId('69f5c615f8b8295dfe3682d5'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing',
    enrolledAt: ISODate('2026-05-02T09:38:29.027Z')
  },
  {
    _id: ObjectId('69f5c615f8b8295dfe3682d6'),
    name: 'Rahul',
    age: 20,
    course: 'Python',
    status: 'completed',
    enrolledAt: ISODate('2026-05-02T09:38:29.027Z')
  },
  {
    _id: ObjectId('69f5c619f8b8295dfe3682d8'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing',
    enrolledAt: ISODate('2026-05-02T09:38:33.247Z')
  }
]
courseDB>

```

8. Use Case: Course Enrollment System

This system represents how students enroll in different courses.

- Students are added when they enroll in a course
- Each record stores details like name, age, course, and status
- Data can be filtered based on course or status
- Course completion is updated using update operations
- Students can be removed if they drop out

This demonstrates how MongoDB can be used in real-world educational platforms.

9. Conclusion

In this assignment, basic MongoDB operations such as insert, read, update, and delete were performed. Query operators were also used to filter and retrieve

specific data.

End of Assignment